

Activity 14 - Design and document a reorder feature with AI

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K5/A5 K4/A4

Specification

`reorder_suggestions(&[StockItem]) -> Vec<Suggestion>`. R1 point = usage x lead + safety; R2 items above the point are excluded; R3 order_qty = point + 7 days usage - on_hand, zero quantities excluded; R4 overflowing items are skipped, never wrapped. Output sorted by SKU.

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain designing with types (newtypes), requirement-named tests and design trade-offs to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is `samples/01_design_with_types.rs` from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on designing with types (newtypes), requirement-named tests and design trade-offs with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles software design and traceability with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: `reorder_suggestions(&[StockItem]) -> Vec<Suggestion>`. R1 point = usage x lead + safety; R2 items above the point are excluded; R3 order_qty = point + 7 days usage - on_hand, zero quantities excluded; R4 overflowing items are skipped, never wrapped. Output sorted by SKU. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Design and document a reorder feature with AI" in `starter/src` so that the supplied tests pass. Give the AI R1-R4 and the public types; require a trace table in its answer and verify each row against the tests. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_design_with_types.rs

Run: rustc --edition 2021 01_design_with_types.rs && ./01_design_with_types

// Sample: newtypes make invalid values hard to create.

```
struct Units(u32);

struct Sku(String);

impl Sku {
    fn parse(text: &str) -> Option<Sku> {
        let valid = text.len() == 8 && text.as_bytes()[3] == b'-';
        valid.then(|| Sku(text.to_string()))
    }
}

fn order(sku: &Sku, qty: Units) -> String {
    format!("order {} x {}", qty.0, sku.0)
}

fn main() {
    match Sku::parse("ABC-0001") {
        Some(sku) => println!("{}", order(&sku, Units(90))),
        None => println!("invalid SKU"),
    }
    println!("bad SKU accepted? {}", Sku::parse("bad").is_some());
}
```

Expected output:

```
order 90 x ABC-0001
bad SKU accepted? false
```

02_trace_with_tests.rs

Run: rustc --edition 2021 --test 02_trace_with_tests.rs && ./02_trace_with_tests

// Sample: tests named after requirements form a living trace matrix.

/// R1: point = usage x lead + safety. R2: above the point, no order.

/// R3: order up to point + 7 days of usage. R4: overflow gives None.

```
pub fn suggest(
    on_hand: u32,
    usage: u32,
    lead: u32,
    safety: u32,
) -> Option<u32> {
    let point = usage.checked_mul(lead)?.checked_add(safety)?;
    if on_hand > point {
        return None;
    }
    point
        .checked_add(usage.checked_mul(7)?)?
        .checked_sub(on_hand)
}

#[cfg(test)]
mod tests {
    use super::*;
```

```

#[test]
fn r1_r3_below_point_orders_up_to_target() {
    assert_eq!(suggest(50, 10, 5, 20), Some(90));
}

#[test]
fn r2_above_point_no_order() {
    assert_eq!(suggest(71, 10, 5, 20), None);
}

#[test]
fn r4_overflow_is_none() {
    assert_eq!(suggest(0, u32::MAX, 2, 0), None);
}
}

```

Expected output:

test result: ok. 3 passed

Learn more

Rust Book: Test Organization: <https://doc.rust-lang.org/book/ch11-03-test-organization.html>

Rust API Guidelines: Documentation: <https://rust-lang.github.io/api-guidelines/documentation.html>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php